Poll

% python # what is your likely next move?

2

import numpy

0%

import torch

0%

import tensorflow

0%

import jax.numpy as np

0%

import matplotlib.pyplot as plt

0%

import scipy

0%

import torchvision or torchaudio

0%

import keras / pytorch-lightning or similar

0%

print("hello world") # this is my first python program

50%

Paste from ChatGPT

50%

Let $x \in \mathbb{R}$ be a scalar, and $p(x)$ is a probability density function. What can we say about the range of $p(x)$?

0

$p(x)$ is monotonic in x

0%

$p(x) \in [0, 1]$

0%

$p(x) \geq 0$

0%

$p(x)$ is convex

0%

Stochastic differential equations

0

I think I've heard of that

0%

Seen and forgotten at least once

0%

Just the basics - like Fokker-Planck

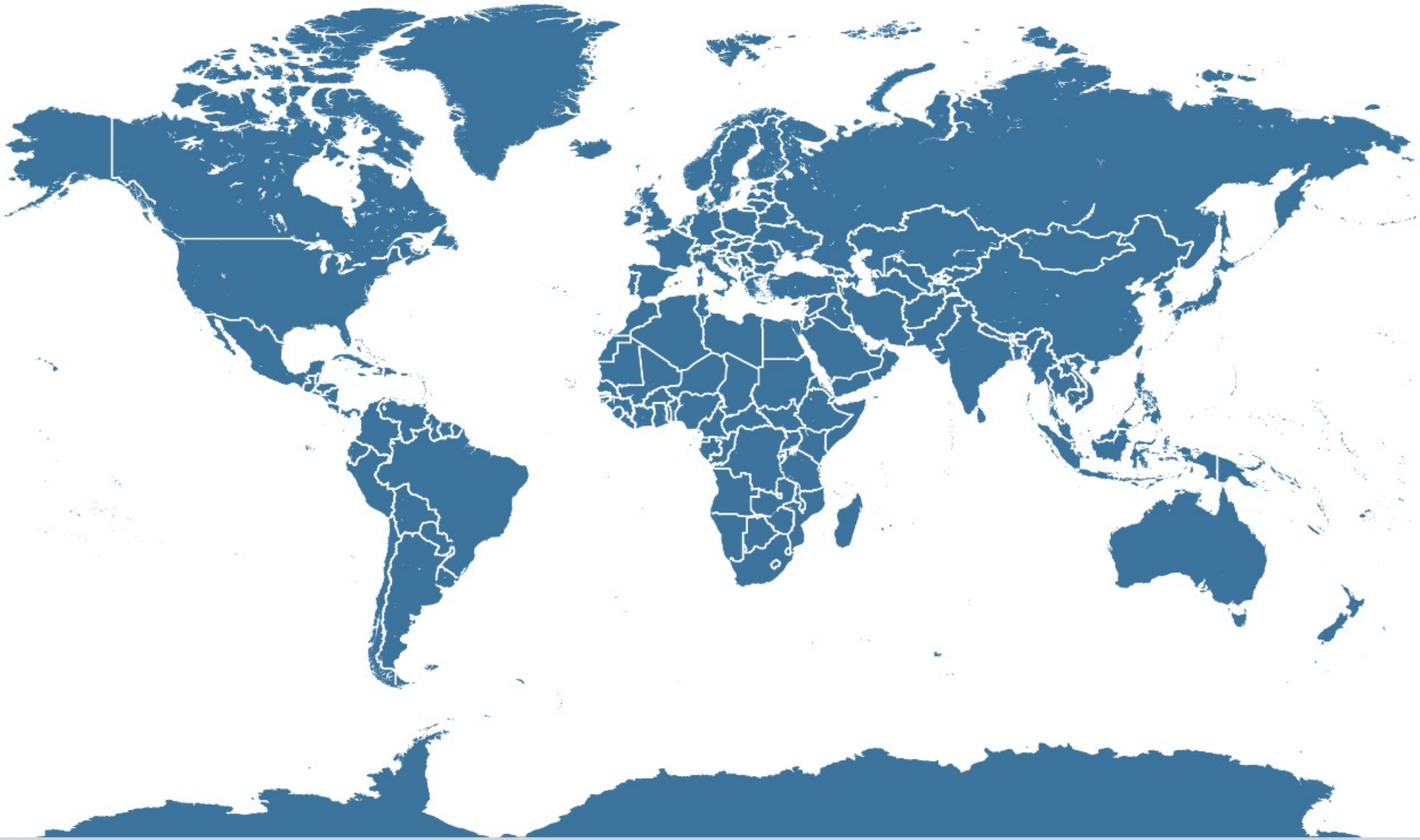
0%

I have an opinion about Ito versus Stratonovich

0%

Where does the best food come from?

0



What do you think is a realistic Artificial General Intelligence timeline?

0

Never
0%

> 50 years, major obstacles with no clear solution using today's methods
0%

10-50 years, more scale and methodological innovations
0%

1-10 years, AGI is imminent on current trajectory
0%

0 years. Already more intelligent than average voter.
0%

Ghost Summoning – Part II

GPT = Ghosts Programming Together

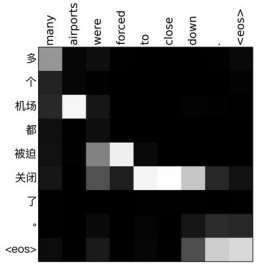
Attention and modified attention

Understanding Key-value-query

References

- As always, great links and visualizations from https://lilianweng.github.io/posts/2018-06-24-attention/
- Some pictures from: https://jalammar.github.io/illustrated-transformer/
- Attention is all you need paper
- BERT paper is good too, of course, and introduces the critical bidirectional masked language model objective
- https://bbycroft.net/llm Cool viz.

Self-attention



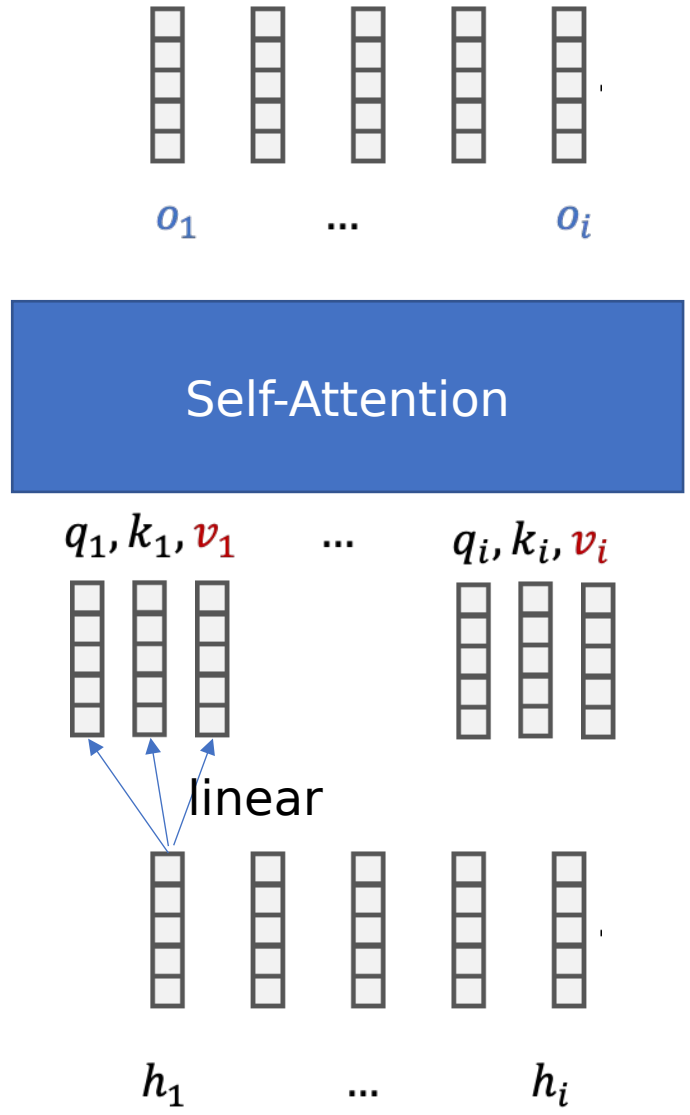
ATTENTION matrix – which positions "attend" to which other positions

("single head" in this case)

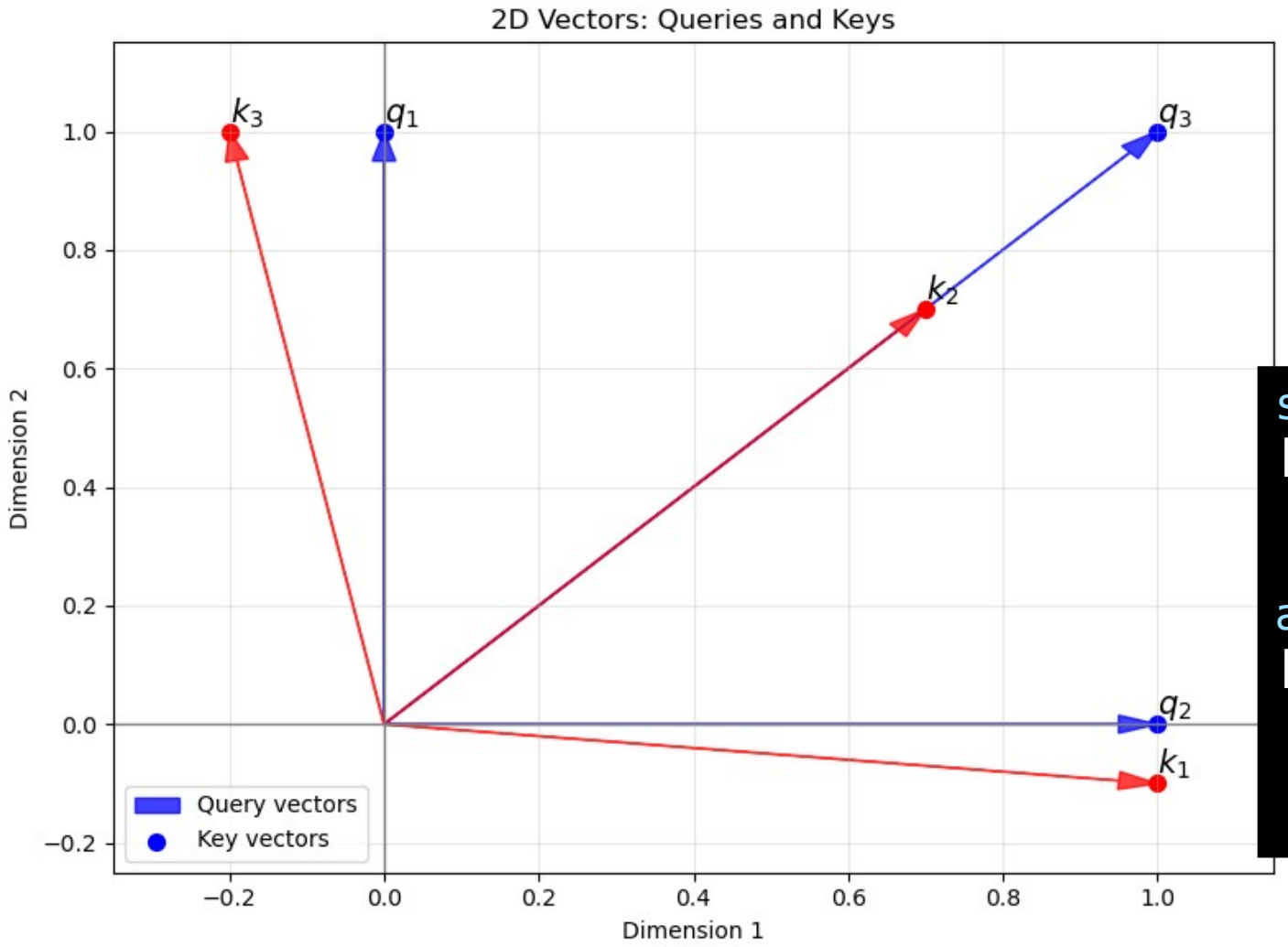
$$p(j|i) = softmax_j(q_i \cdot k_j) = \frac{e^{q_i \cdot k_j}}{\sum_{j'} e^{q_i \cdot k_{j'}}}$$

Output depends weights "values" by attention

$$o_i = \sum_j p(j|i) v_j$$



2-d attention viz (bi-directional, like BER

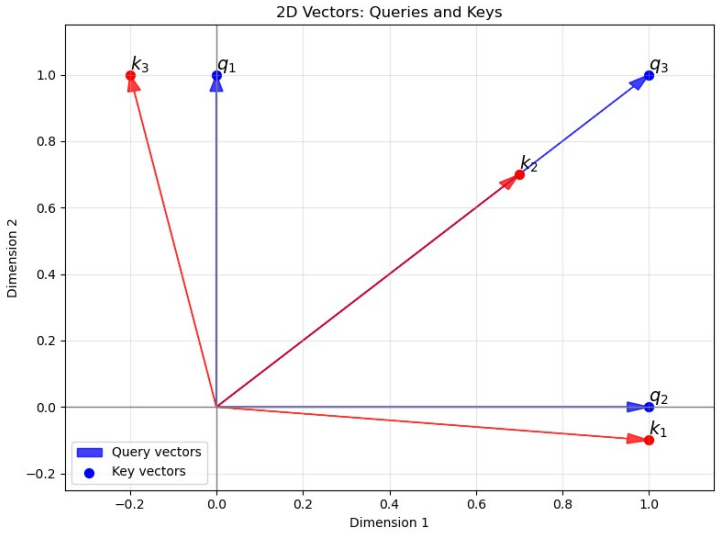


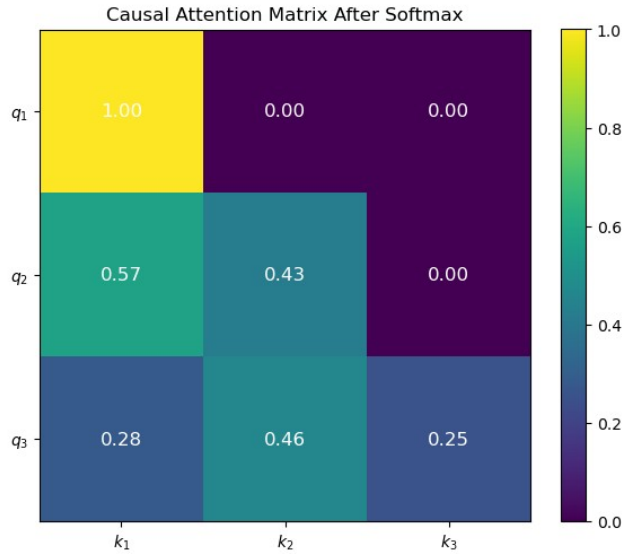

```
q = torch.tensor([
[0.0, 1.0], # q1: up
[1.0, 0.0], # q2: right
[1.0, 1.0], # q3: up-right])

k = torch.tensor([
[1.0, -0.1], # k1: right
[0.7, 0.7], # k2: up-right
[-0.2, 1.0], # k3: mostly up,])
```

```
scores = q @ k.T
[[-0.10,  0.70,  1.00],  #q1
 [ 1.00,  0.70, -0.20],  #q2
 [ 0.90,  1.40,  0.80]]  #q3
attn = torch.softmax(scores, dim=-1)
[[0.16, 0.36, 0.48],   #q1
 [0.49, 0.36, 0.15],   #q2
 [0.28, 0.46, 0.25]]   #q3
   #k1   #k2   #k3
```

Causal attention






```
scores = q @ k.T

[[-0.10,  0.70,  1.00],
 [ 1.00,  0.70, -0.20],
 [ 0.90,  1.40,  0.80]]

Causal mask?
```

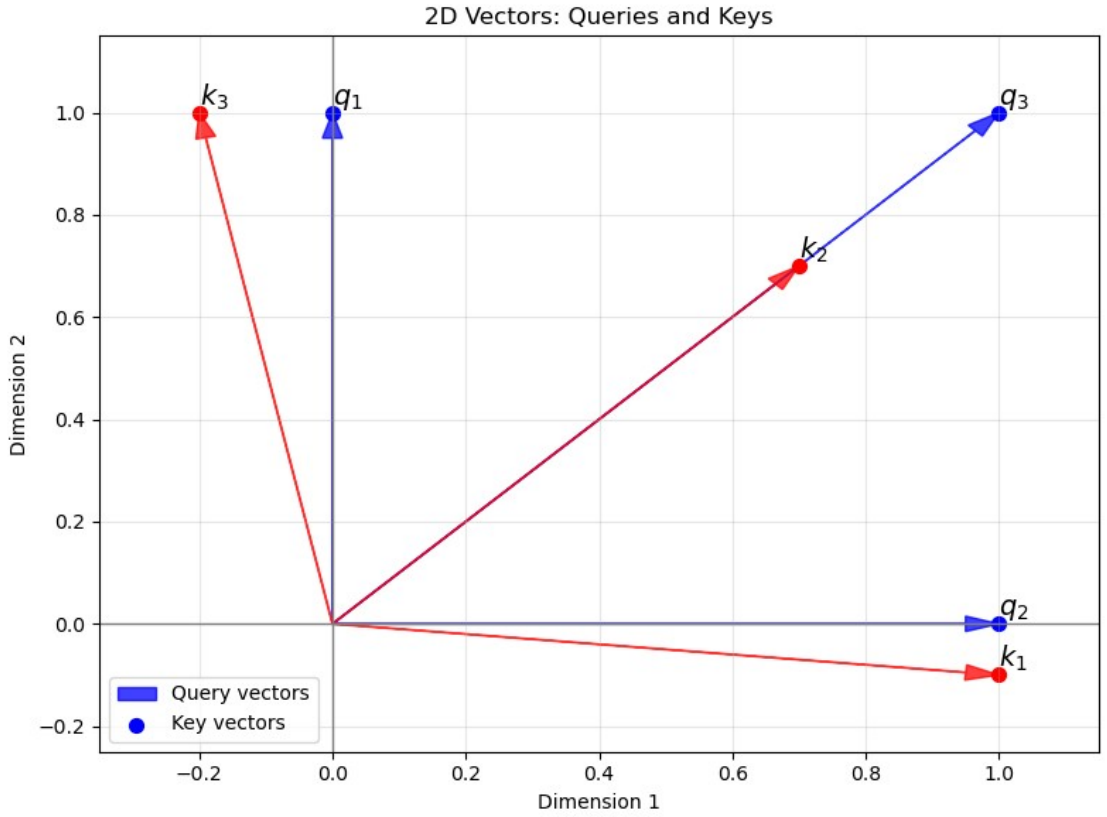
```
masked_scores =
[[-0.10,  -inf,  -inf],  #q1
 [ 1.00,  0.70,  -inf],  #q2
 [ 0.90,  1.40,  0.80]]  #q3
   #k1        #k2        #k3
```

```
attn = torch.softmax(masked_scores, dim=-1)
[[1.00, 0.00, 0.00],
 [0.55, 0.45, 0.00],
 [0.30, 0.42, 0.28]]
```

Look at softmax and scaling

```
# Not let's get the "dot products" of all pairs
qk_dot = q @ k.T
print(qk_dot)  # check the size
```

```
tensor([[-0.89, -0.66, -0.20],
        [ 0.14,  0.33,  0.23],
        [ 0.14,  0.47,  0.35]])
```




```
torch.softmax(qk_dot, axis=1)
```

```
tensor([[0.24, 0.30, 0.47],
        [0.30, 0.37, 0.33],
        [0.28, 0.38, 0.34]])
```

```
torch.softmax(qk_dot/100, axis=1)
```

```
tensor([[0.33, 0.33, 0.33],
        [0.33, 0.33, 0.33],
        [0.33, 0.33, 0.33]])
```

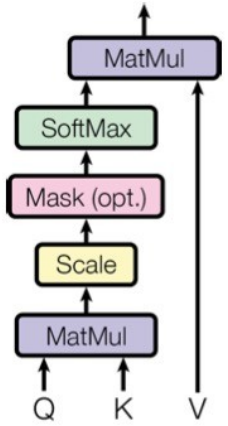
```
torch.softmax(qk_dot*100, axis=1)
```

```
tensor([[    0.00,     0.00,     1.00],
        [    0.00,     1.00,     0.00],
        [    0.00,     1.00,     0.00]])
```

NanoGPT code (but “FlashAttn” recommended)

```
# manual implementation of attention
att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
att = att.masked_fill(self.bias[:,:,:T,:T] == 0, float('-inf'))
att = F.softmax(att, dim=-1)
att = self.attn_dropout(att)
y = att @ v # (B, nh, T, T) x (B, nh, T, hs) -> (B, nh, T, hs)
```

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

Example call to flash attention

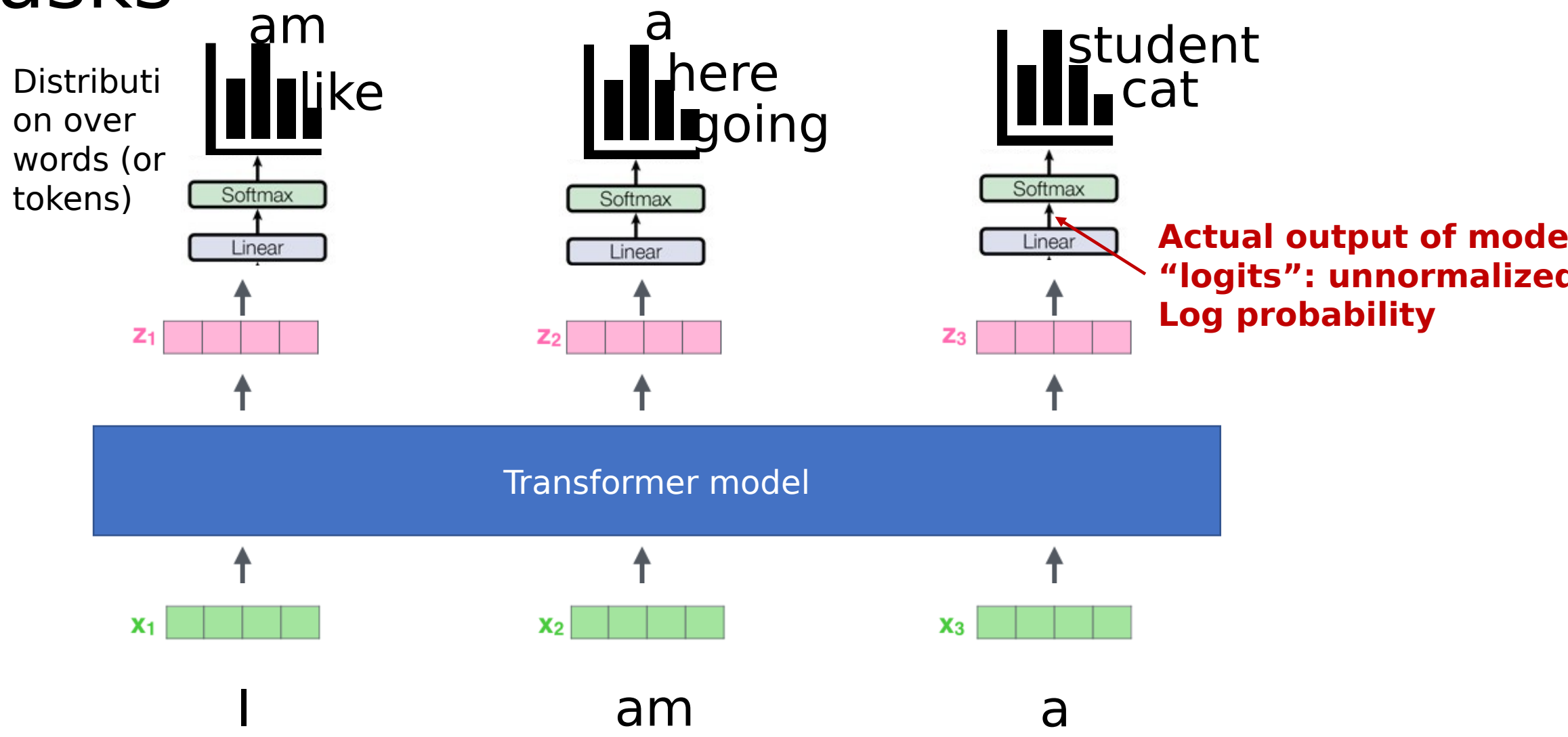
```
# If using rotary position
embedding, apply here
q, k = apply_rope(q,k)

out = flash_attn_func(
    q, k, v,
    dropout_p=0.0,
    causal=False
)  # softmax(QK^T)V
```

- You give it Q, K, V
- It does the matrix multiplication, efficiently, in high bandwidth GPU memory
- (Apparently it can deal with different length sequences, without padding!)

Ways to use Attention

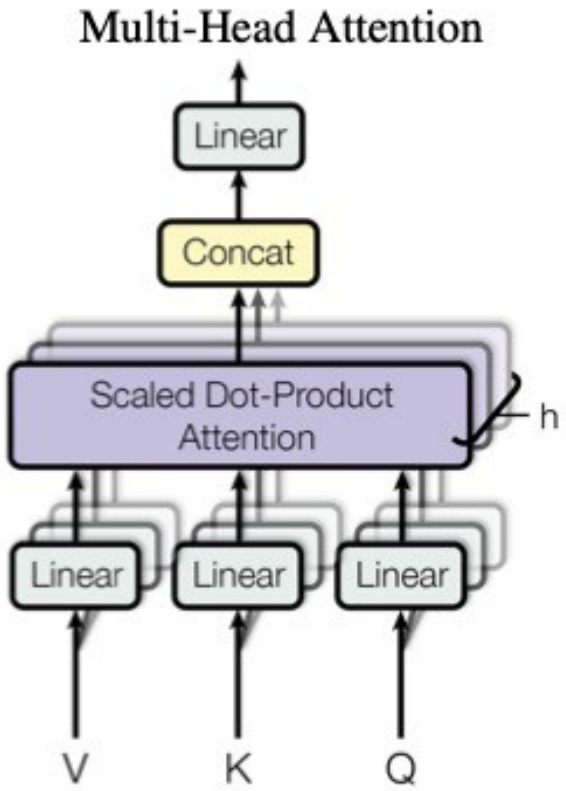
“Prediction heads” for different tasks



Three types of transformer layer

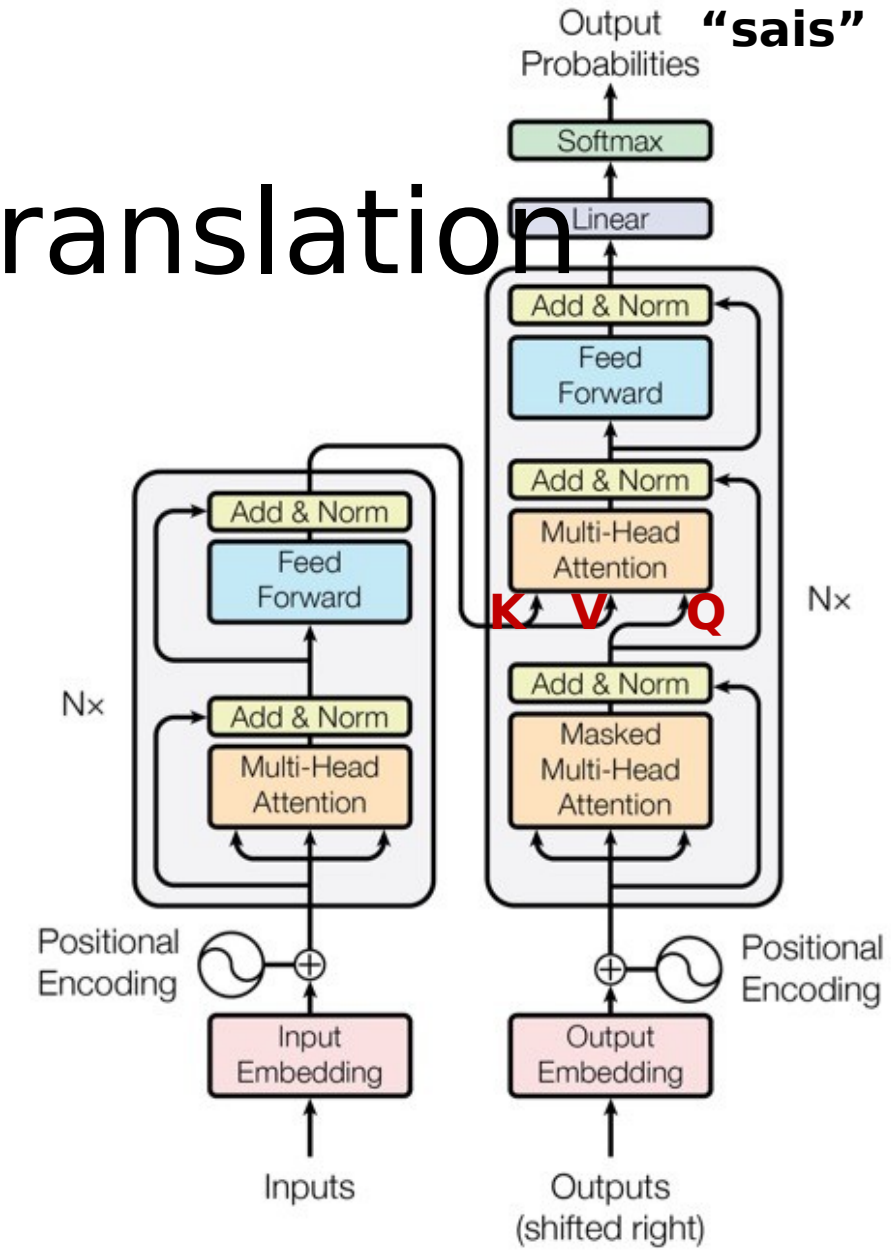
- Encoder-decoder (Translation)
- Encoder (BERT)
- Decoder (GPT)

Original Transformer encoder-decoder for translation



Original transformer:

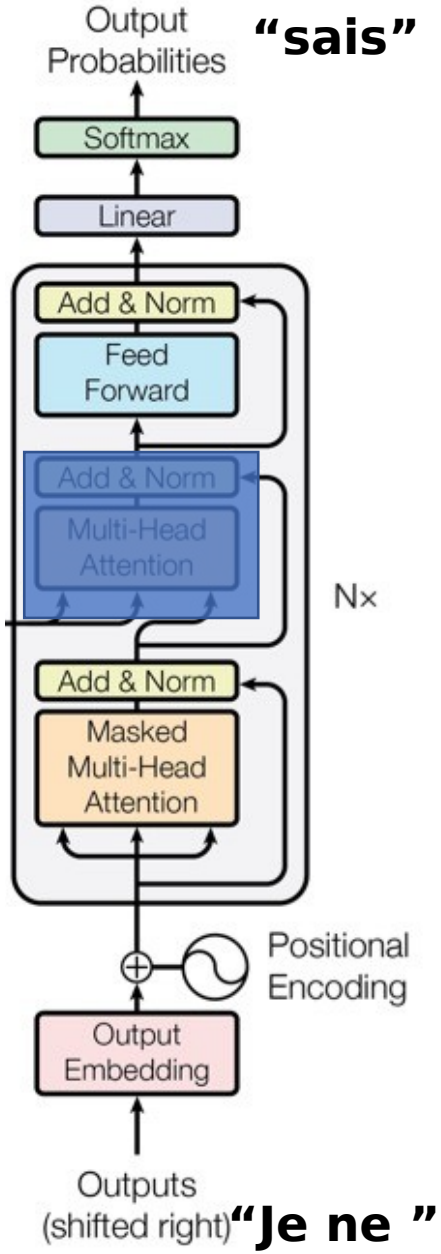
- Motivated by translation
- Produces one token at a time
- Has encoder/decoder structure



(Also T5 model from Google)

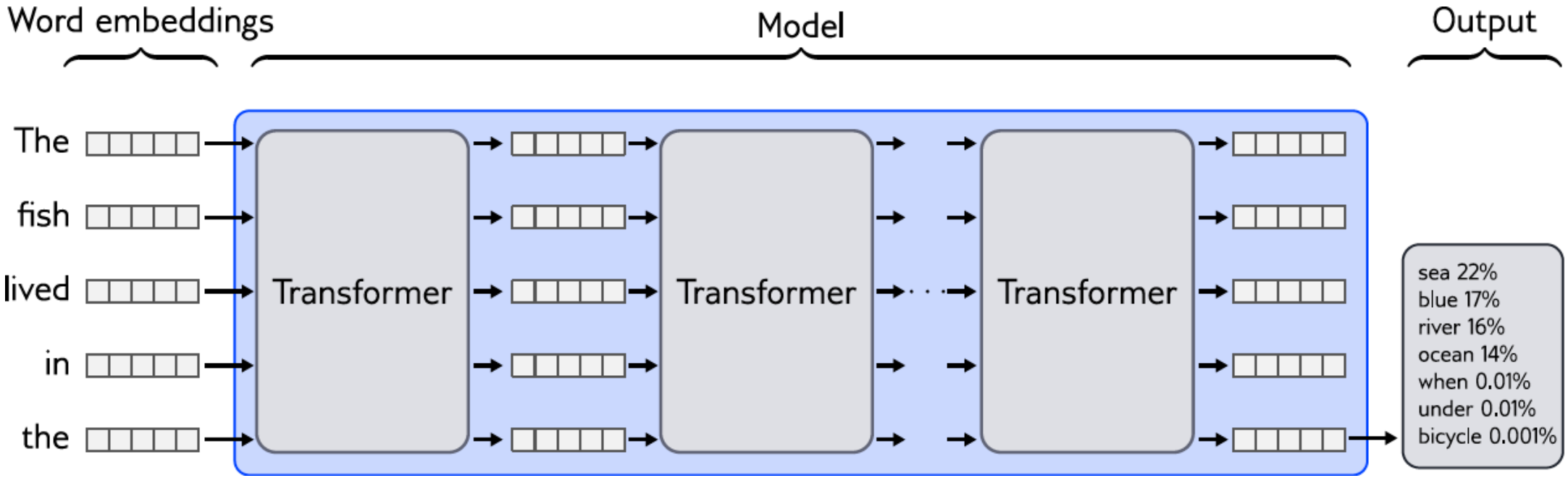
GPT: “Decoder-only”

- Motivated by text generation (also popular for classification)
- Produces one token at a time
- Has “decoder” only

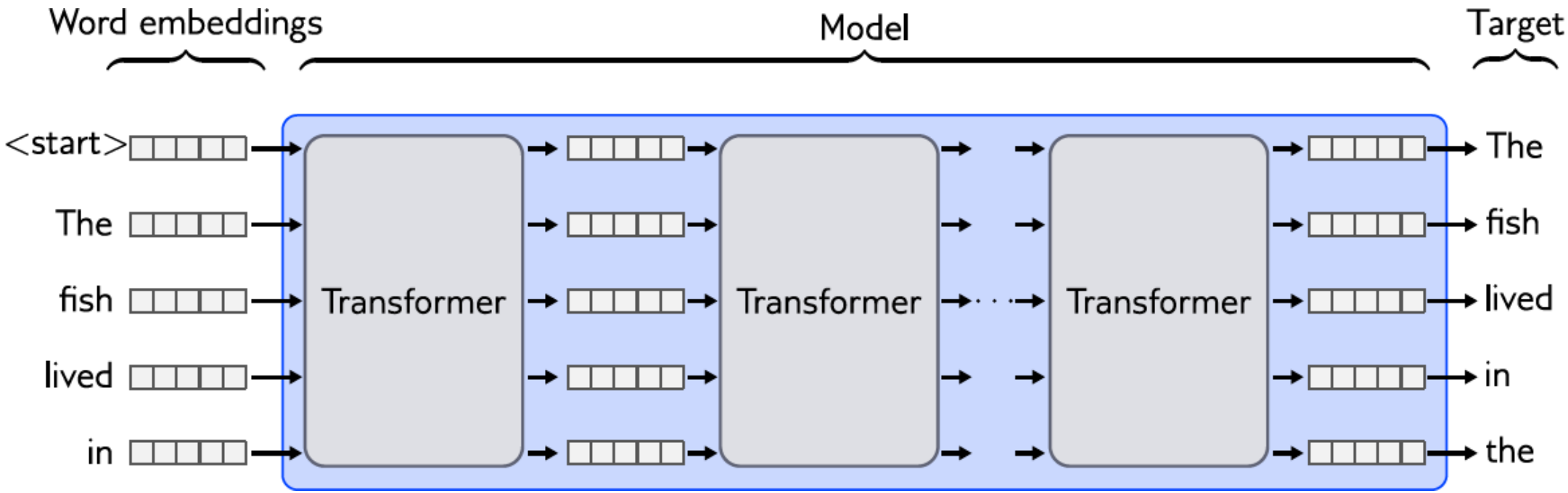


[“Improving language understanding by generative pre-training”]

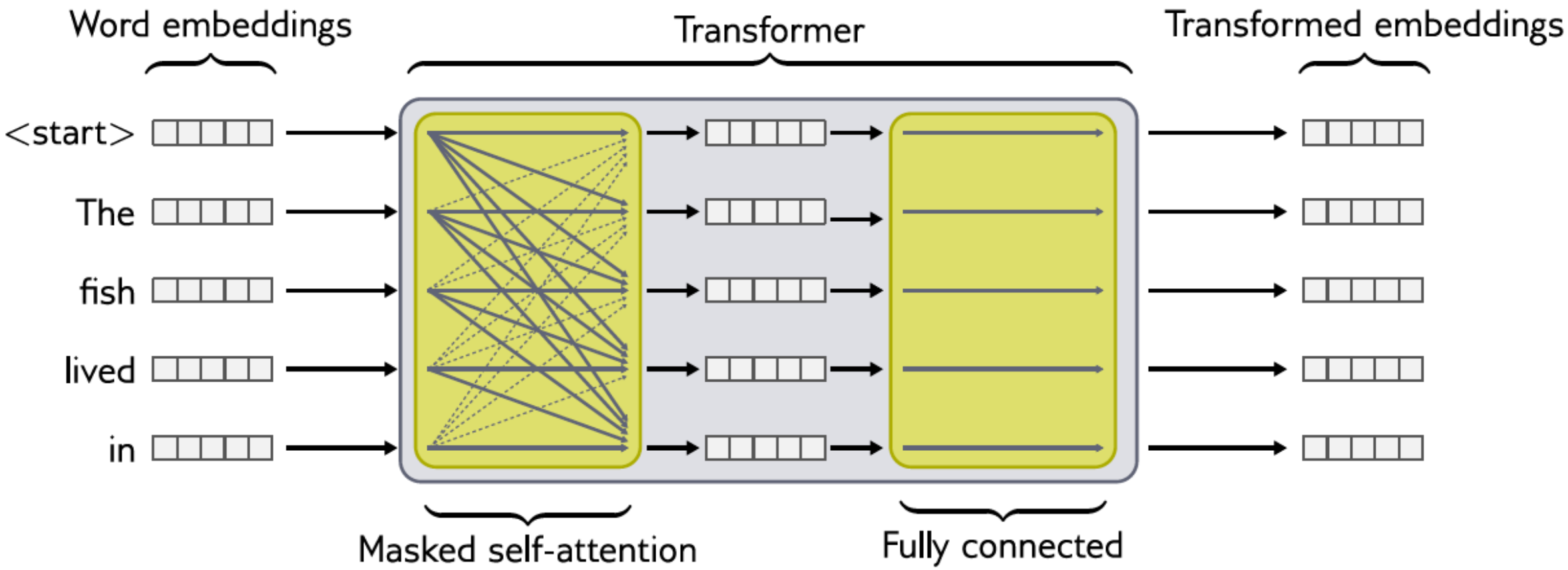
Predicting next - Generation



Predicting all next words simultaneously - Training

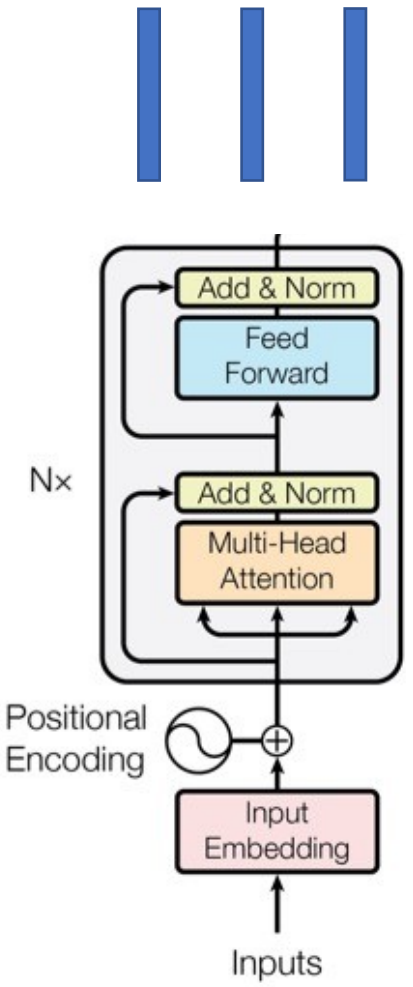


Masked self-attention



BERT: “Encoder-only”

Sequence of embeddings



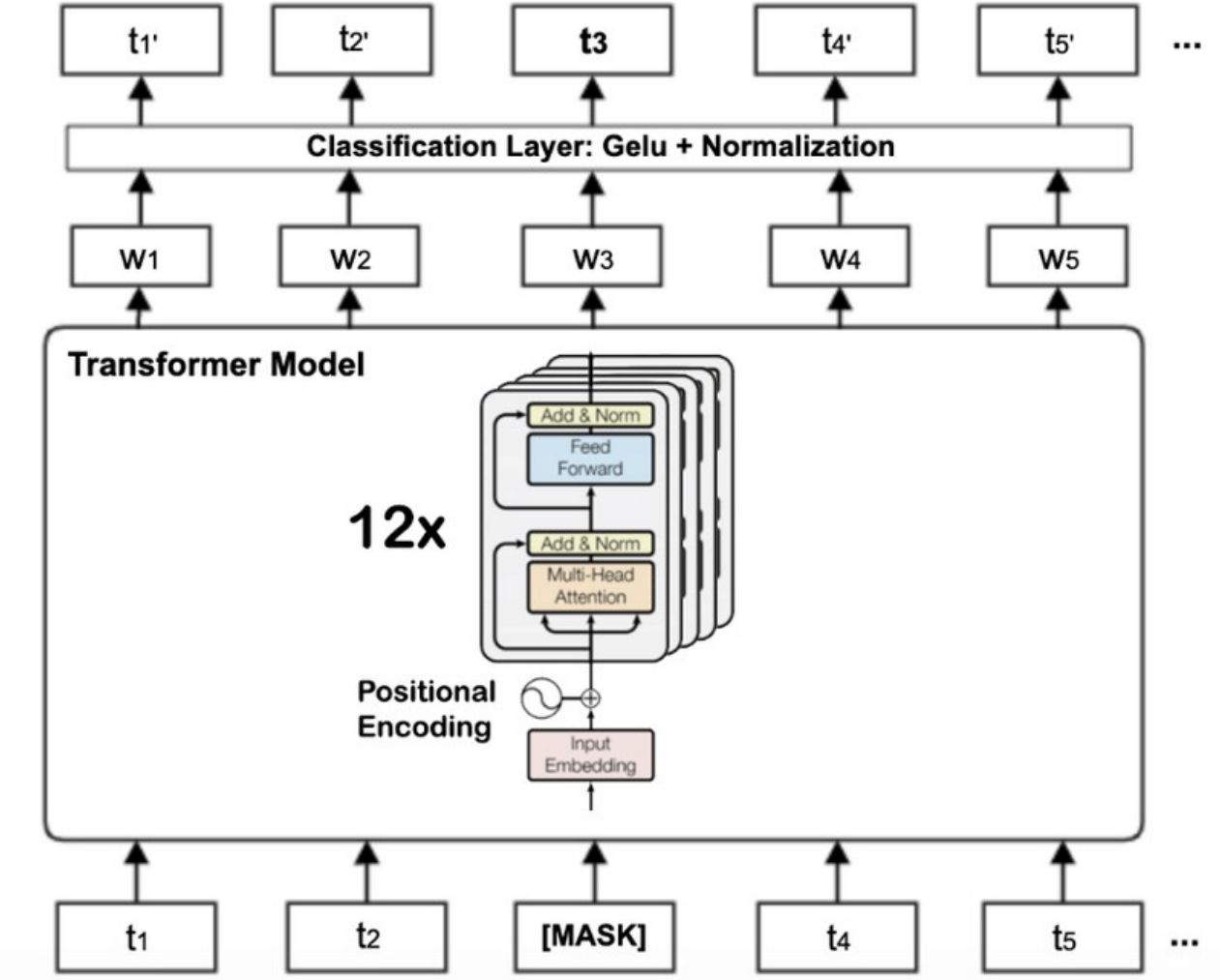
“I don’t know”

https://arxiv.org/abs/1810.04805

BERT (“Encoder Only”, “Bidirectional attention”)

- Predict all words in a sequence at once
- not a probability model.
- trained to predict masked words. (“Masked language modeling” versus “causal language modeling”)

Predict p(t3 | everything else)
But this is not an autoregressive probability mode
for the whole sequence



Preferred (?) for:

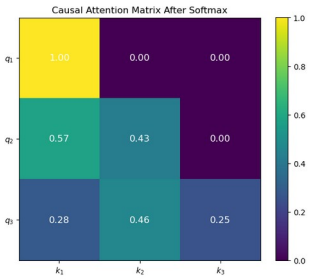
- **Sentence embedding**
- Classification
- Named entity recognition

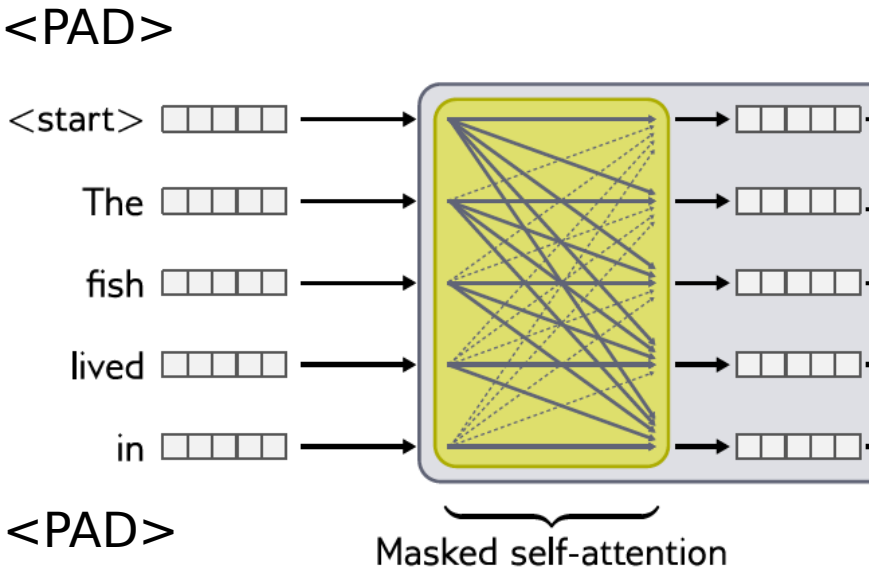
Faster inference

“Diffusion transformers” are a popular, modern “encoder only” architecture

Modifying attention

Attention masking

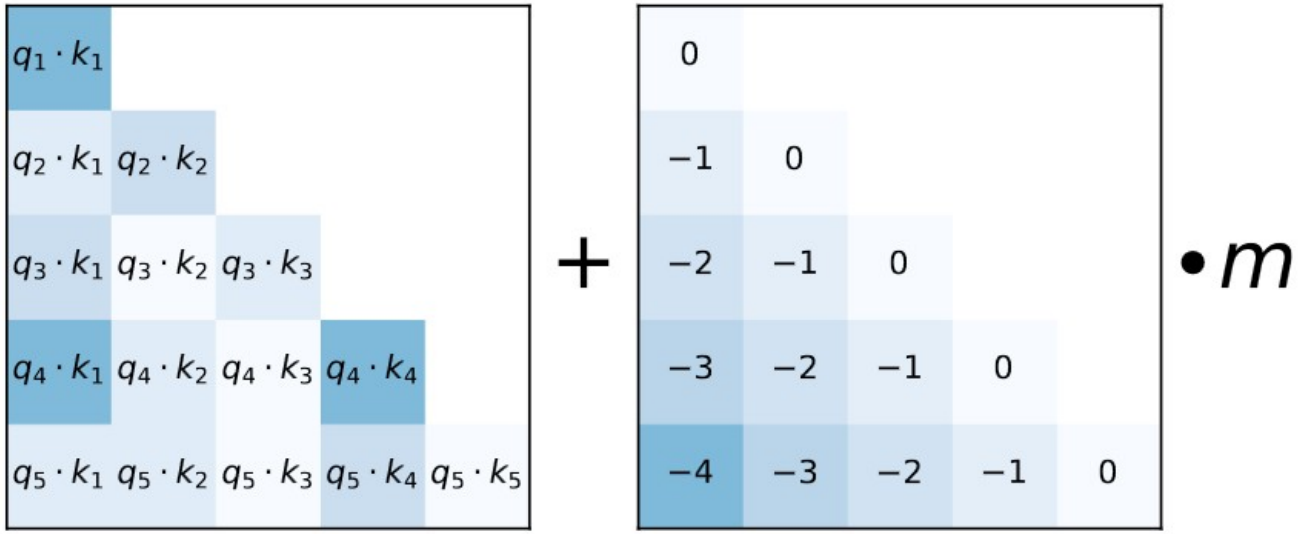




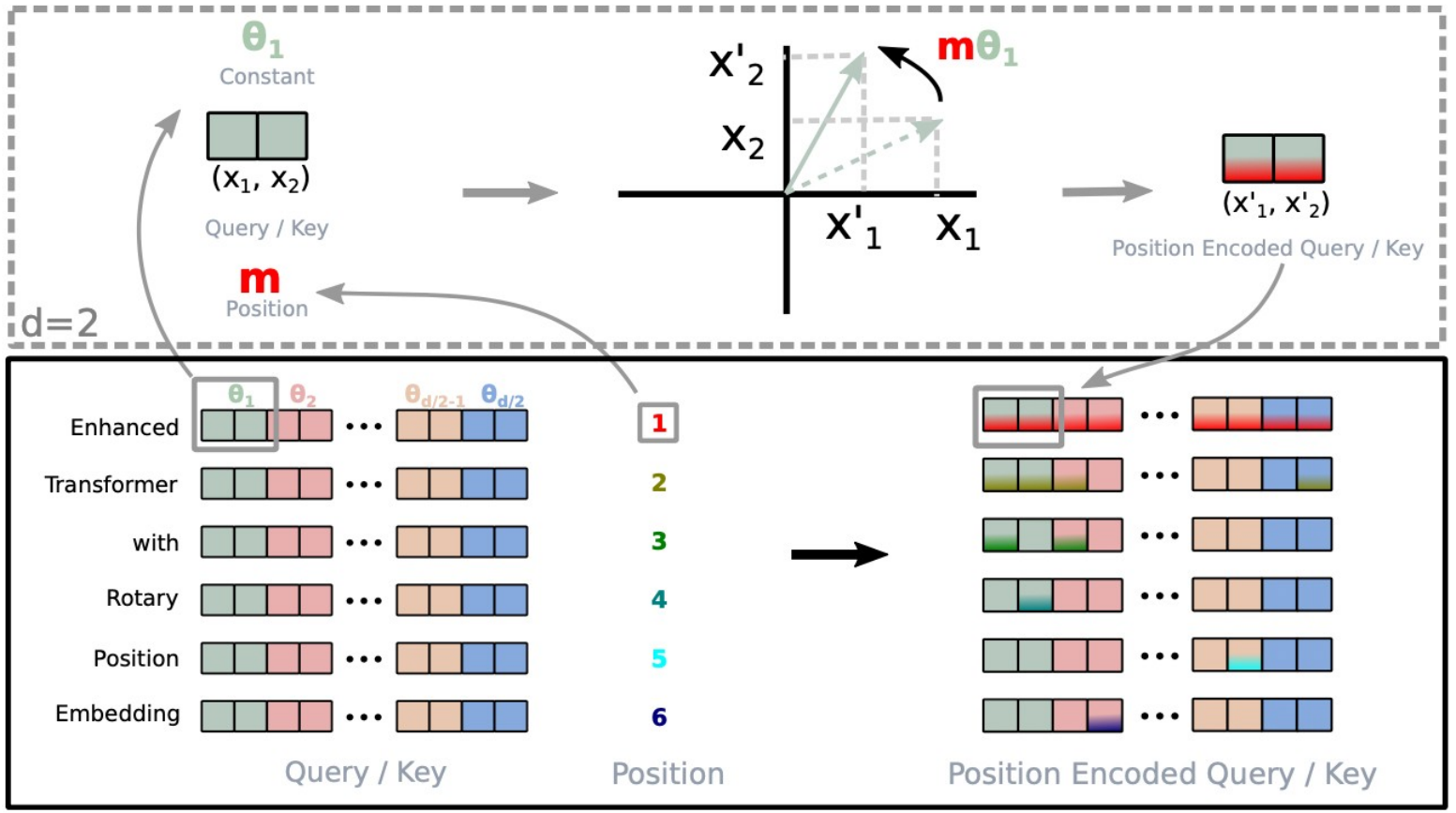
- For **Causal** LMs, we attend only to tokens before the current token
- We also ignore **Padding tokens** using attention masks.

We can also modify the attention for other purposes…

Attention with linear biases... another approach to position embedding (ALIBI)



Rotation based position embedding?



https://arxiv.org/abs/2104.09864

RoPE: apply transformation to key/query, so that the dot product will depend on *relative* position

- Want only dot-product attention to only depend on relative positions:

$$\boldsymbol{q}_m^T \boldsymbol{k}_n = \left\langle f_q\left(\boldsymbol{x}_m, m\right), f_k\left(\boldsymbol{x}_n, n\right)\right\rangle = g\left(\boldsymbol{x}_m, \boldsymbol{x}_n, m-n\right)$$

ake inspiration from complex numbers (Board)

- Rotate the pre-activations instead of adding:
- $\boldsymbol{q}_m = \mathbf{R}^d_{\Theta,m}\mathbf{W}_q\boldsymbol{x}_m$
- $\boldsymbol{k}_n = \mathbf{R}^d_{\Theta,n}\mathbf{W}_k\boldsymbol{x}_n$
-

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

Aside on attention and CLIP?

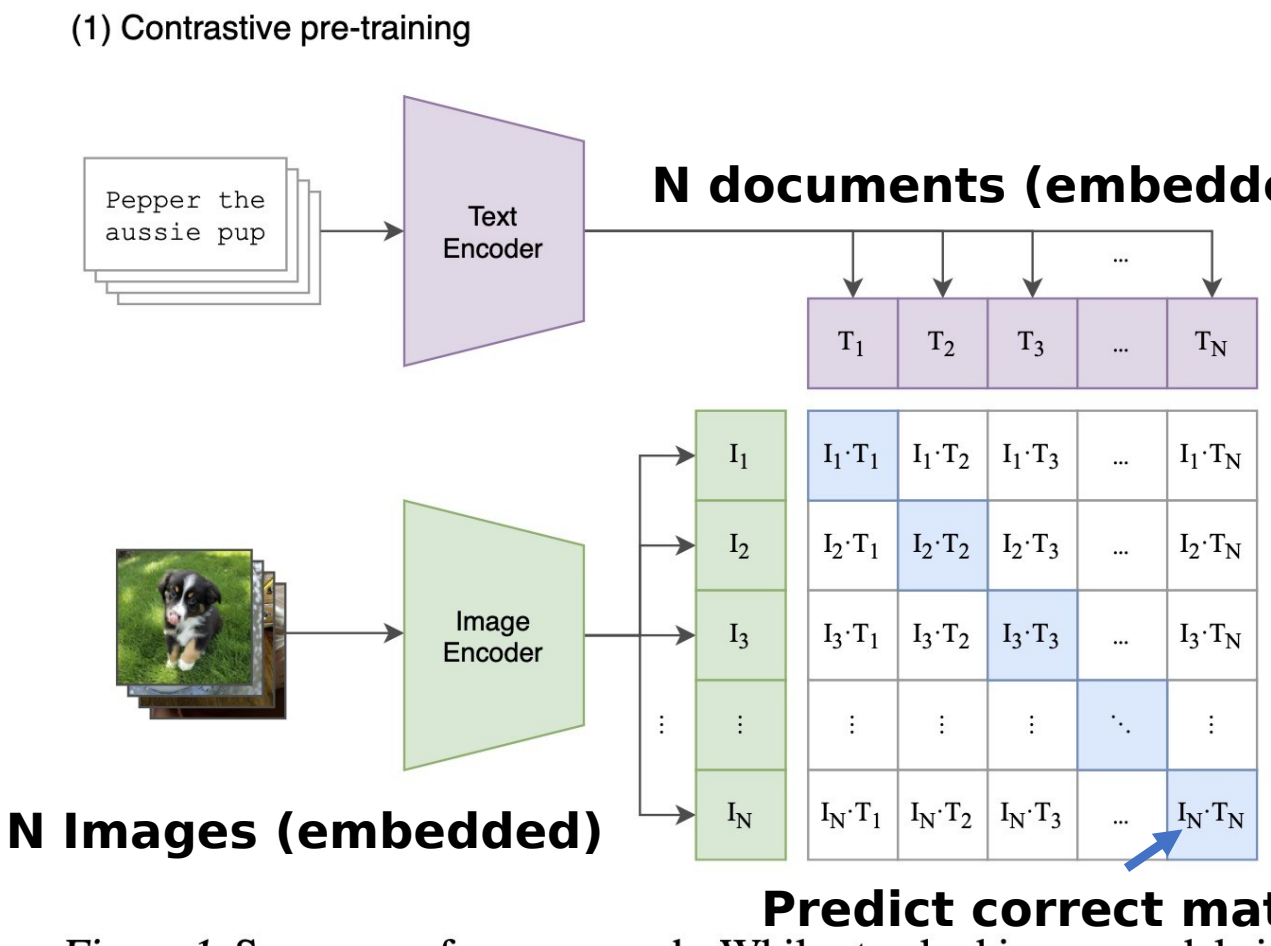
If time

Attention and CLIP

"Attention" looks identical to contrastive / self-supervised pretraining used in methods like CLIP for text/image relationships

$$p(text\ i|image\ j) = \frac{e^{\boldsymbol{T}_i \cdot \boldsymbol{I}_j}}{\sum_{i'} e^{\boldsymbol{T}_{i'} \cdot \boldsymbol{I}_j}}$$

Then use cross entropy to train on true image-text pairs.



$$p(i|j) = \frac{e^{T_i \cdot I_j}}{\sum_{i'} e^{T_{i'} \cdot I_j}}$$

From pairs of (image, text),
(I1, T1), (I2, T2)…
Use cross entropy loss to try to predict that each “image” attends to correct “text”.

Figure 1. Summary of our approach. While standard image models jointly train an image feature extractor and a linear classifier to predict some label, CLIP jointly trains an image encoder and a text encoder to predict the correct pairings of a batch of (image, text) training examples. At test time the learned text encoder synthesizes a zero-shot linear classifier by embedding the names or descriptions of the target dataset’s classes.

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

Live code demo

Call up a Qwen model

Look at the tokenizer / options needed for generation, training

Look at outputs (logits, KV cache, attention)